

Teste de Software

Roteiro *Sprint 1* - confronto: ia versus sua especificação

Objetivo: Analisar criticamente o código gerado pela ferramenta de IA a partir da sua Questão 8 (Semana 09-05-2026), identificando o que a ferramenta acertou, o que omitiu e o que produziu incorretamente para o contexto do internet banking.

Passo 1. Analise sua resposta da Questão semana 09-05-2026 e copie o texto completo.

Questão 8. Você estudou por que automatizar, quando não automatizar, o que é TDD, o que é um framework e quais são as boas práticas. Nesta semana, no laboratório de informática. Descreva com precisão técnica: qual teste você escreveria primeiro para o sistema de internet banking usando a abordagem TDD? Inclua: o cenário escolhido, o caso de borda que o teste cobre, o que o teste deve verificar ANTES do código ser escrito e qual categoria de framework você utilizaria. Sustente cada escolha no texto lido e cite as páginas.”

Resposta esperada (RAHMAN et al., 2024, p. 52; BSTQB, 2023, p. 52):

Primeiro teste: verificação de transferência com saldo suficiente. Abordagem TDD (RAHMAN et al., 2024, p. 52): escrever o teste antes da função - o teste define que `transfer(conta_origem, conta_destino, valor)` deve retornar sucesso quando `saldo >= valor`.

Caso de borda: saldo exatamente igual ao valor da transferência (condição de fronteira - conecta com técnica de caixa-preta vista nas semanas anteriores.

O que o teste verifica antes do código: a especificação do comportamento esperado, não a implementação. Categoria de framework: ferramenta de execução de testes unitários (BSTQB, 2023, p. 52).

Passo 2. No AVA, faça o download do arquivo `app.py` e coloque na pasta principal do projeto “internet_banking_tests”

Passo 3. Abra o servidor Flask através do arquivo `app.py` no VSCode que está no ambiente virtual `venv`. (`python app.py`)

Passo 4. Abra o GitHub Copilot Chat no VS Code (`Cmd+Shift+I` no macOS / `Ctrl+Alt+I` no Windows). Abra um novo arquivo chamado “`test_ia_gerado.py`” no editor, **COLE O PROMPT ABAIXO** no campo de chat da GitHub Copilot substituindo `[SUA_Q8]` pelo texto copiado no Passo 1 e pressione Enter:

Gere testes pytest para a seguinte especificação de testes de um sistema de internet banking. A API Flask está disponível em <http://127.0.0.1:5000> com os endpoints:

GET /saldo/<conta_id>

POST /transferencia - corpo: {"origem":int,"destino":int,"valor":float}

GET /extrato/<conta_id> Especificação: [SUA_Q8]

Requisitos obrigatórios:

- Inclua um teste para o caso de borda em que o saldo da conta de origem é exatamente igual ao valor da transferência.
- Marque em comentário (# Primeiro teste TDD) - primeiro teste segundo a abordagem TDD descrita na especificação.

Gere apenas o código Python. Sem explicações adicionais.

Com o servidor Flask já rodando no Terminal 1 (Passo 3), não feche esse terminal.

- Abra um segundo terminal no VSCode (Ctrl+Shift+ `), ative a mesma venv e instale as dependências de teste (uma única vez por máquina):

Windows:

```
.\venv\Scripts\activate  
pip install pytest requests
```

Linux/macOS:

```
source venv/bin/activate  
pip install pytest requests  
pytest test_ia_gerado.py -v
```

Resultado esperado – primeira vez que rodar o pytest:

```
● (venv) MacBook-Pro-de-Claudinei:internet_banking_tests claudinei$ pytest test_ia_gerado.py -v  
===== test session starts =====  
platform darwin -- Python 3.13.2, pytest-9.0.3, pluggy-1.6.0 -- /Users/internet_banking_tests/venv/bin/python3.13  
cachedir: .pytest_cache  
hypothesis profile 'default'  
rootdir: /Users/internet_banking_tests  
plugins: hypothesis-6.152.7, cov-7.1.0, bdd-8.1.0  
collected 12 items  
  
test_ia_gerado.py::test_saldo_conta_existente PASSED [ 8%]  
test_ia_gerado.py::test_saldo_conta_inexistente PASSED [ 16%]  
test_ia_gerado.py::test_transferencia_saldo_igual_valor PASSED [ 25%]  
test_ia_gerado.py::test_transferencia_saldo_suficiente PASSED [ 33%]  
test_ia_gerado.py::test_transferencia_saldo_insuficiente PASSED [ 41%]  
test_ia_gerado.py::test_transferencia_valor_negativo PASSED [ 50%]  
test_ia_gerado.py::test_transferencia_valor_zero PASSED [ 58%]  
test_ia_gerado.py::test_transferencia_origem_destino_iguais PASSED [ 66%]  
test_ia_gerado.py::test_transferencia_campos_ausentes PASSED [ 75%]  
test_ia_gerado.py::test_transferencia_conta_origem_inexistente PASSED [ 83%]  
test_ia_gerado.py::test_extrato_conta_existente PASSED [ 91%]  
test_ia_gerado.py::test_extrato_conta_inexistente PASSED [100%]  
  
===== 12 passed in 1.18s =====
```

Pergunta para investigação. Execute o comando `pytest test_ia_gerado.py -v` uma segunda vez (consecutivas), sem reiniciar o servidor Flask entre elas. Na primeira execução, os doze testes passam. Na segunda, o `test_transferencia_saldo_igual_valor` (PRIMEIRO TESTE TDD) falha com status HTTP 422 - saldo insuficiente. Investigue, sustentando as respostas em referências do material da disciplina:

a) que decisão do `app.py` viabiliza essa diferença de comportamento entre execuções consecutivas?

b) que característica de "bom caso de teste" (ISTQB CTFL v4.0.1) está sendo violada pelos testes gerados pela IA?

c) essa violação estava prevista na sua resposta da Q8 ou foi uma omissão tanto sua quanto da ferramenta?

Passo 5. Leia o arquivo gerado linha a linha. Preencha a tabela abaixo:

Critério técnico	Presente?	Linha(s)	Observação
Cenário principal da Q8 coberto			
Caso de borda identificado na Q8			
Teste de saldo insuficiente			
Teste de conta inexistente (404)			
Teste de valor negativo ou zero			
Teste de mesma conta origem/destino			
Fixture de isolamento (setup/teardown)			
Verificação de saldo após transferência			

Passo 6. Para cada "Não" na tabela: documente porque esse cenário é relevante no domínio bancário e qual consequência a omissão pode gerar em produção.

Passo 7. Compare o output da IA com sua Q8. Registre: o que a IA gerou que você não havia descrito; o que você descreveu e a IA não gerou; o que a IA gerou diferente da sua especificação.

Reflexão do sprint — registre em seu arquivo Word antes de avançar

Responda com suas próprias palavras. Não existe resposta certa. O que importa é que o texto revele o seu raciocínio real durante a execução.

1. Antes de ver o código gerado, você esperava que a IA do GitHub cobrisse tudo o que você especificou na Q8? O que te surpreendeu - para mais ou para menos?
2. Qual foi a diferença mais relevante entre o que você descreveu e o que a IA produziu? Por que essa diferença importa para um sistema bancário real?
3. Com base no que você leu em COUTINHO; NASCIMENTO (2025), o comportamento da IA que você observou é esperado ou inesperado? Cite a página.
4. Em uma linha: qual é o limite fundamental da IA nesse tipo de tarefa?

Referências Bibliográficas

BSTQB — BRAZILIAN SOFTWARE TESTING QUALIFICATIONS BOARD. Certified Tester Foundation Level Syllabus: versão 4.0. [S. l.]: ISTQB; BSTQB, 2023. Capítulo 6, p. 52–53. Disponível em: https://www.bstqb.online/files/syllabus_ctfl_4.0br.pdf. Acesso em: maio 2026.

COUTINHO, N. de M.; NASCIMENTO, E. L. B. do. Desafios e benefícios da implementação de testes automatizados em empresas de software. Cuadernos de Educación y Desarrollo, v. 17, n. 4, e8221, p. 1-19, 2025. Disponível em: <https://ojs.cuadernoseducacion.com/ojs/index.php/ced/article/view/8221>. Acesso em: maio 2026.